Design of TM5-700 Intelligent Manipulator Based on LLM and Visual Processing

Report : WEI FENG

Data: 2026/6/3

TM5-700 + tool camera

Background:
From manually set actions, it has moved towards language understanding, visual perception and automatic planning.

Manually set

Traditional robotic arms rely on fixed scripts, coordinates and action sequences for their tasks.

Language Entry

Natural language lowers the operational barriers for non-professional users.

Visual state

The tool camera enables the system to obtain the actual position of the blocks on the desktop.

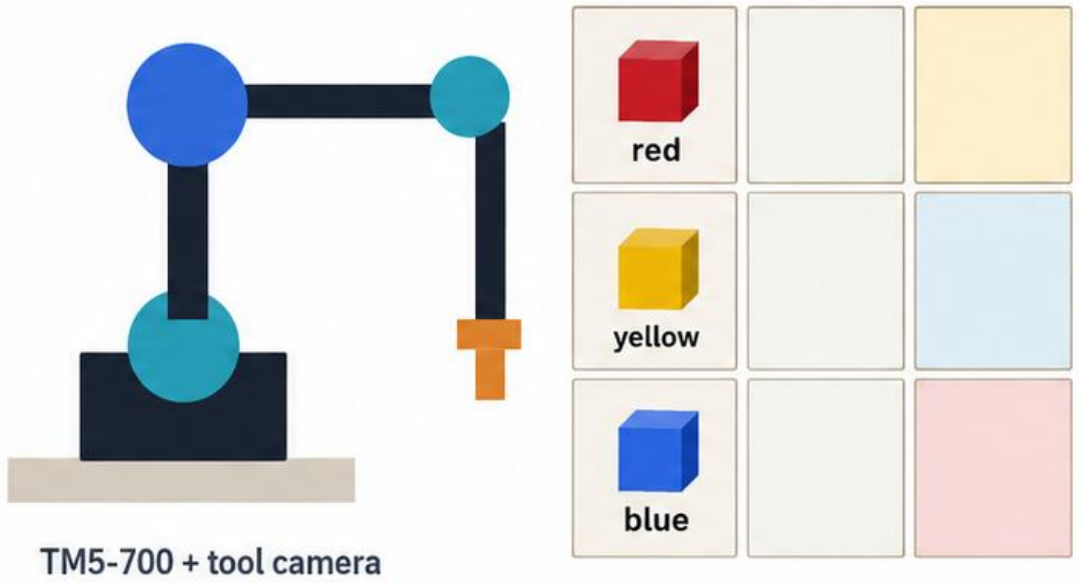
Mission planning

The planner converts the target state into a sequence of executable actions.

This project builds a system that can understand natural language, sense the desktop status, and automatically plan the movements of the robotic arm.

TASK SCENE

The target scenario is abstracted as color-block picking and sorting on a 3×3 tabletop grid.



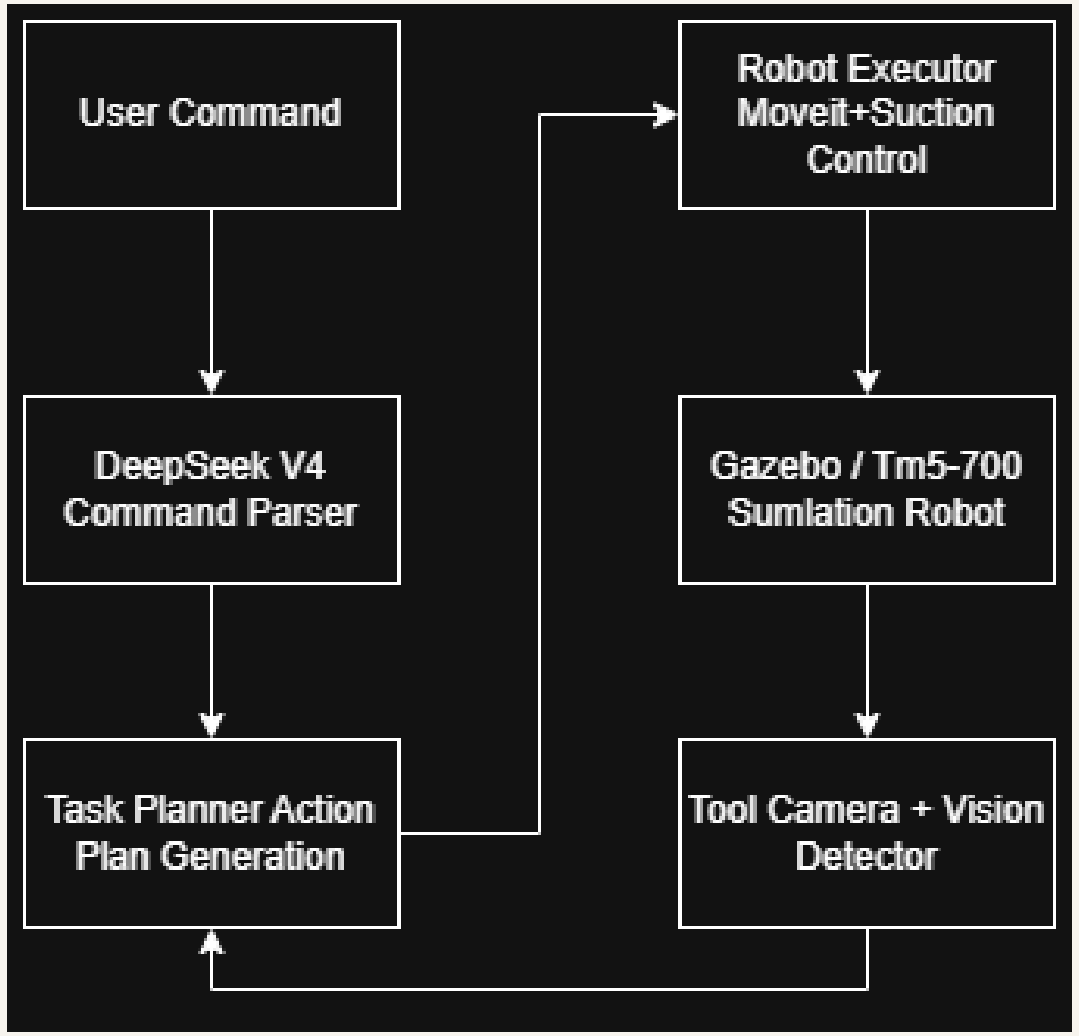
Gazebo simulation: TM5-700 + tool camera + pick table

Goal	Natural-Language-driven color-block sorting system
Environment	ROS 2 Humble + Gazebo + MoveIt 2
Hardware	TM5-700 robot arm, tool camera, pick table
Objects	red / yellow / blue cubes
Workspace	3×3 logical grid

- **Core task:** recognize the current state, generate the target state, and complete pick, move, place, and verification.

System Design: Neither LLM nor vision directly controls the robotic arm; the planner serves as the safety boundary between semantics and execution.

1. LLM and Vision do not directly control the robotic arm.
2. The planner is responsible for generating the sequence of actions.
3. The actuator only executes the verified action_plan.



C

RRTConnect for TM5-700: Simplified Formula View

Only the essential equations are kept: robot state, valid space, and local extension.

Planning Logic

1. **Two Trees**
   Grow from start and goal
2. **Sample**
   Pick q_rand in joint space
3. **Extend**
   Move one step toward sample
4. **Check**
   Joint limits + collision
5. **Connect**
   Output feasible waypoints

Output: a collision-free sequence of joint waypoints, later time-parameterized by MoveIt.

Essential Equations + Key Parameters

1 Robot state

$$q = [q_1, q_2, q_3, q_4, q_5, q_6]^T$$

Parameters

q	6D joint vector
q_i	i-th joint angle

2 Valid configuration space

$$C_{free} = \{q \mid q_{min} \le q \le q_{max},\ \text{Collision}(q) = \text{false}\}$$

Parameters

C_free	valid space
q_min/q_max	joint limits
Collision(q)	collision test

3 Local extension

$$q_{new} = q_{near} + \alpha(q_{rand} - q_{near})$$
$$\alpha = \min\left(1, \eta / d(q_{near}, q_{rand})\right)$$

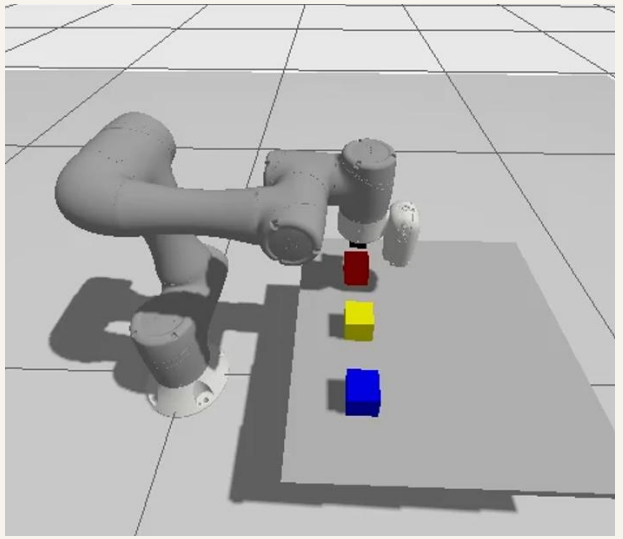
Parameters

q_near	nearest node
q_rand	random sample
q_new	new node
α / η	step ratio / limit

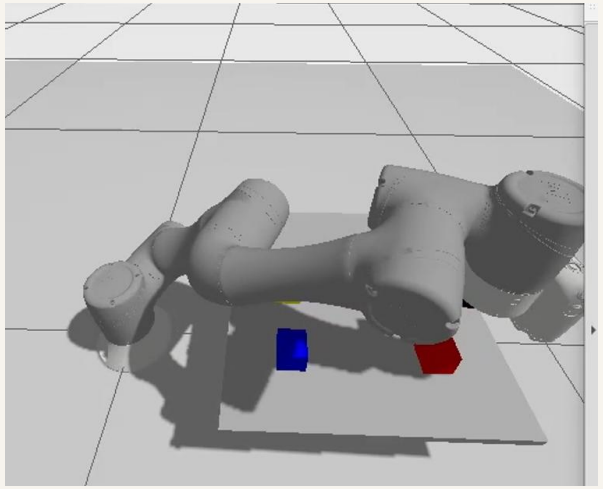
Note: RRTConnect is implemented by OMPL. The TM5-700 MoveIt package provides the robot model, IK, collision checking, and trajectory execution interface.

Gazebo Manipulation Challenge: Gripper Abstraction

- There is no directly available official TM5 fixture model
- The simulation of real contact-type grippers is prone to instability.
- The project adopts suction-based grasping abstraction.
- Gazebo DetachableJoint
  + /suction/attach_color
  +/ suction/detach_color



Attach/red cube



Detach/red cube

joint-settle check

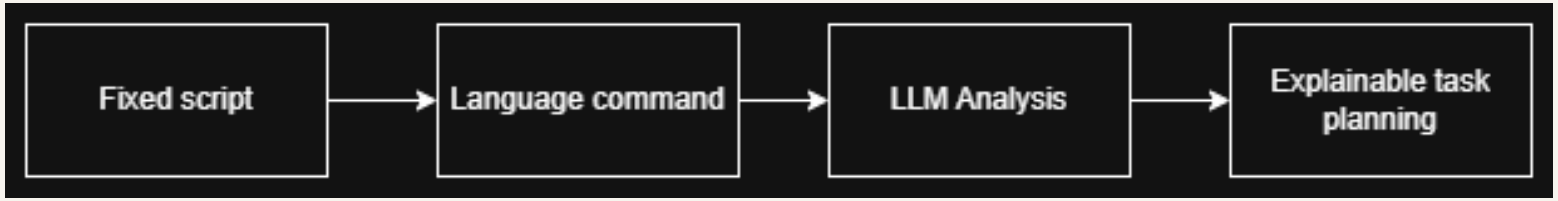
1. The actual joint angle error <= joint_settle_tolerance_rad
2. Joint speed <= joint_velocity_tolerance_rad_s
3. Continuously meet multiple samples
4. Only after being satisfied can the attachment/detachment be carried out.

C

DEMO EVOLUTION

The system has gradually expanded from fixed actions to natural language parsing and task planning.

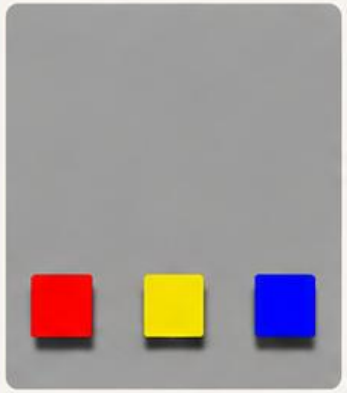
Stage	Ability	Verification function
demo_1	Fix the red square and move it.	Verify the basic pick-and-place execution link
demo_2	Rule parsing single-block instruction	Implement simple Chinese position command
demo_3	LLM Multi-step Instruction Parsing	Support multi-language, multi-color and multi-stage mobile tasks
demo_4	Task Planner - Algorithm Implementation	Handling target occupation, swapping, buffering, and area targets



Vision Pipeline

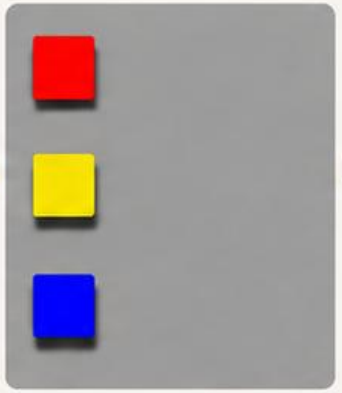
From a fixed ready-view, the image is rotated 90° clockwise. Then HSV segmentation, morphological filtering, contour detection and grid localization are performed.

1 Ready Pose (Captured Image)



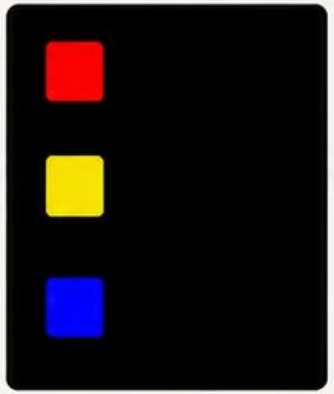
Raw image from the fixed ready pose.

2 Rotate Image 90° Clockwise



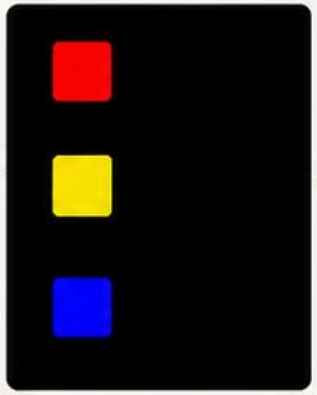
Rotate the image 90° clockwise to align with the observer's view.

3 HSV Color Segmentation (Mask)



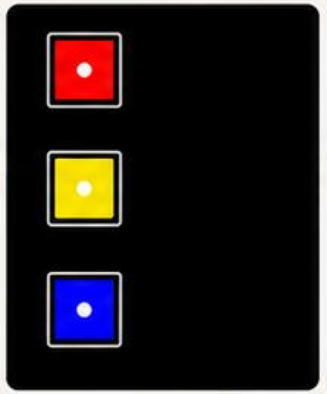
Extract red, yellow and blue regions using HSV thresholding.

4 Morphological Filtering (Mask)



Apply morphological open and close to remove noise and smooth the regions.

5 Contour & Center Extraction



Find the largest contour for each color and compute the centroid (white dot).

6 3×3 Grid Localization

Map the centers to the 3×3 logical grid (aligned with observer's view).

KEY PROCESSING STEPS

- HSV color thresholding (Red / Yellow / Blue)
- Morphological open / close to remove noise
- Largest contour area filtering
- Compute centroid of each contour
- Map centers to the 3×3 logical grid

DETECTED CENTERS → 3×3 LOGICAL GRID MAPPING

Detected Center (Color)	Grid Position (Observer View)	Logical Grid Cell
Top (Red)	Left Top	left_top
Middle (Yellow)	Left Middle	left_middle
Bottom (Blue)	Left Bottom	left_bottom

Note

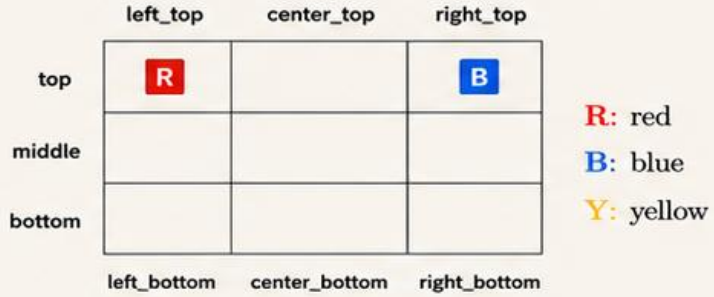
The image is rotated 90° clockwise so that the processing view is consistent with the observer's viewpoint. All subsequent steps are performed in this aligned view.

Task Planning Algorithm: Determine Action Order Based on Occupancy Relations

Core Rule: If target(A) = current(B), then A depends on B. We must move B first, and then move A.

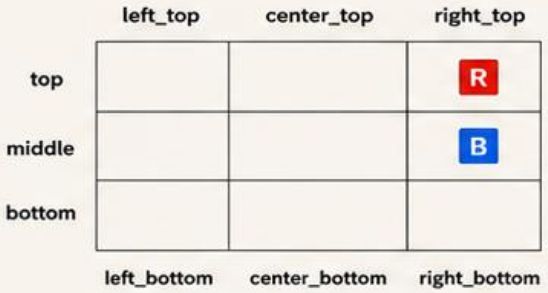
1. Input: Current State and Target State

Current State S_{cur}



red = left_top
blue = right_top

Target State S_{tar}



red = right_top
blue = right_middle

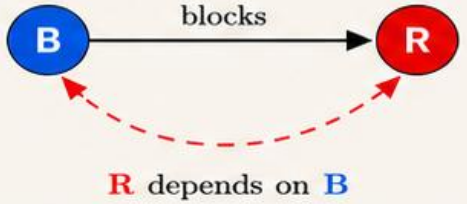
2. Occupancy Relation Judgment

Check whether target(A) = current(B)

- target(red) = right_top
- current(blue) = right_top

Since target(red) = current(blue), red depends on blue.
Therefore, we must move blue first, and then move red.

Dependency Graph



Solid arrow: B blocks R
Dashed arrow: R depends on B (move B before R)

3. Output: Action Plan (Action Sequence)

Step 1: Move blue (remove the blocker from the target cell)

move blue: right_top → right_middle

Step 2: Move red (the target cell is now free)

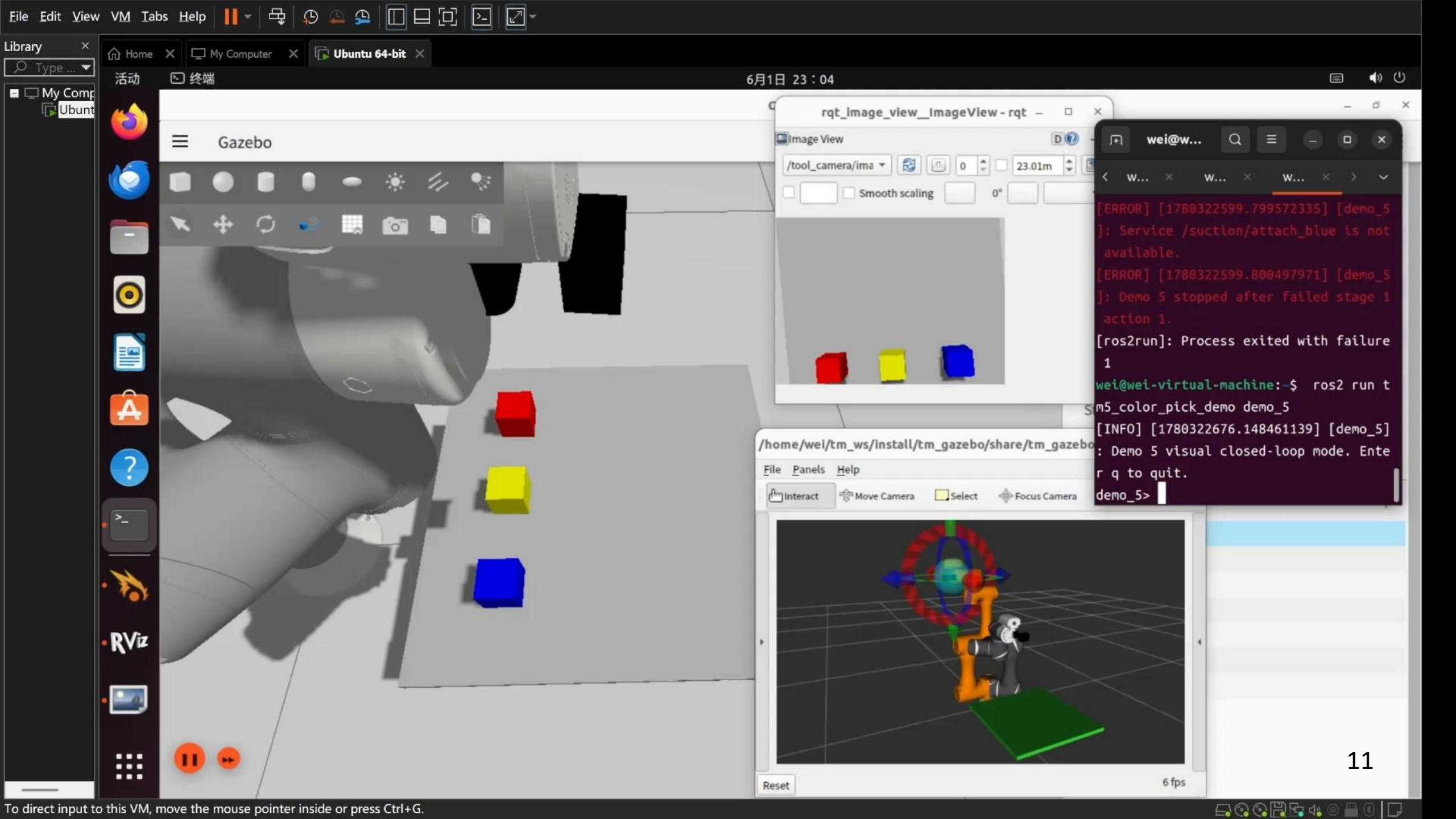
move red: left_top → right_top

Final Action Plan

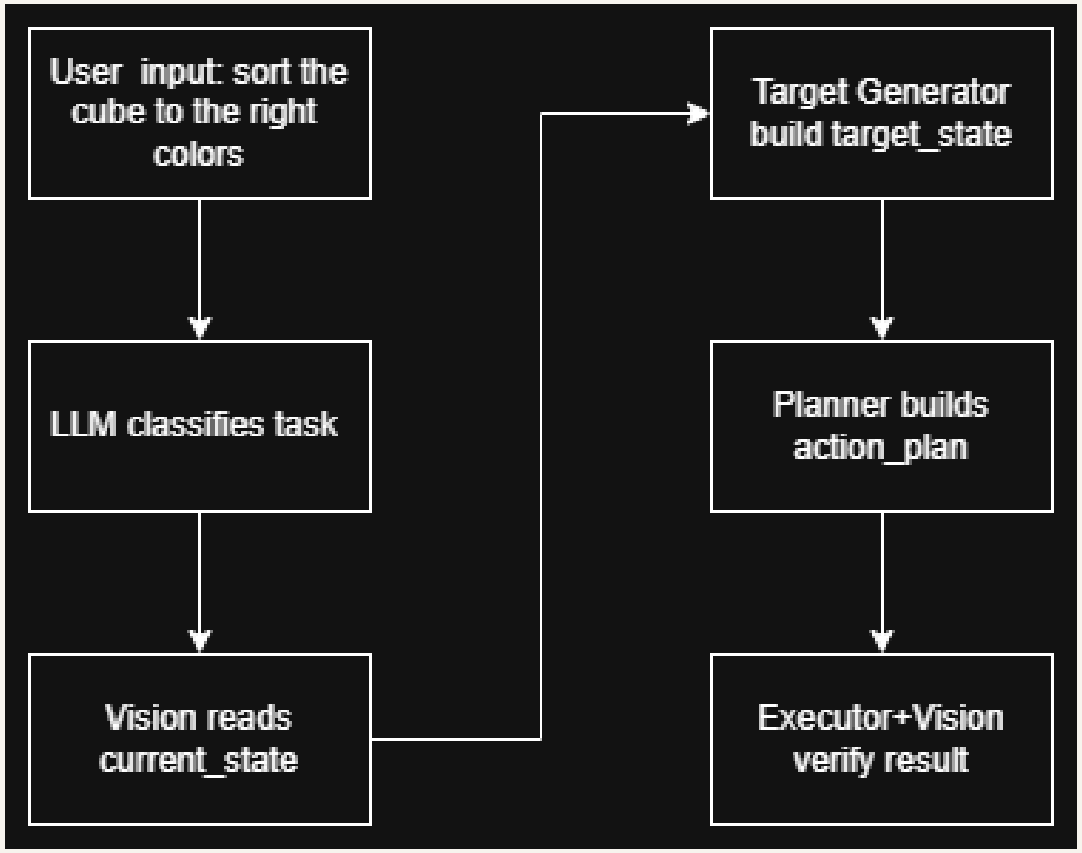
1. move blue: right_top → right_middle
2. move red : left_top → right_top

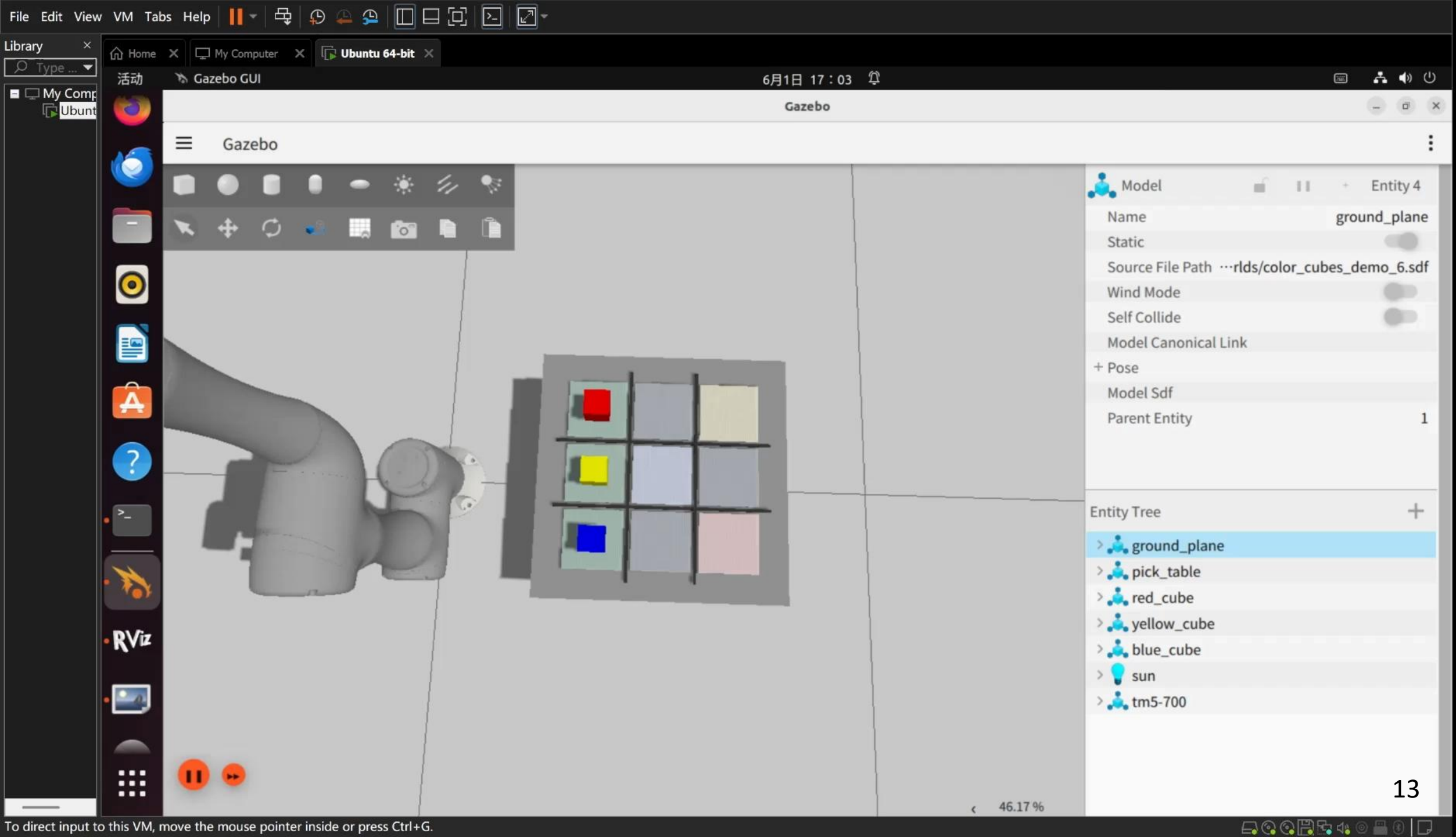
Key Features

- Based on occupancy relation logical and interpretable
- Ensures correct action order and avoids conflicts
- Suitable for discrete grid manipulation tasks
- Not directly controlled by LLM more reliable and safer


File Edit View VM Tabs Help
Library
Home
My Computer
Ubuntu 64-bit
活动
终端
6月1日 23：04
Gazebo
rqt_image_view__ImageView - rqt
Image View
/tool_camera/ima
Smooth scaling
wei@w...
[ERROR] [1780322599.799572335] [demo_5]: Service /suction/attach_blue is not available.
[ERROR] [1780322599.800497971] [demo_5]: Demo 5 stopped after failed stage 1 action 1.
[ros2run]: Process exited with failure 1
wei@wei-virtual-machine:~$ ros2 run tm5_color_pick_demo demo_5
[INFO] [1780322676.148461139] [demo_5]: Demo 5 visual closed-loop mode. Enter q to quit.
demo_5>
/home/wei/tm_ws/install/tm_gazebo/share/tm_gazebo
File Panels Help
Interact
Move Camera
Select
Focus Camera
Reset
6 fps
RViz
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Demo 6: Understanding Abstract Classification Tasks and Generating Targets




File Edit View VM Tabs Help
Library
Home
My Computer
Ubuntu 64-bit
活动
Gazebo GUI
6月1日 17：03
Gazebo
Gazebo
Model
Entity 4
Name
ground_plane
Static
Source File Path
rlds/color_cubes_demo_6.sdf
Wind Mode
Self Collide
Model Canonical Link
Pose
Model Sdf
Parent Entity
1
Entity Tree
ground_plane
pick_table
red_cube
yellow_cube
blue_cube
sun
tm5-700
46.17 %
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

DEMO 5 VS DEMO 6

The value of Demo 6 is upgrading explicit position commands into abstract task semantics.

Comparison Item	demo_5	demo_6
User Command	Explicit target position	Abstract classification task
Example	“Put the red block in the upper-right cell”	“Sort the blocks by color”
Target State	Specified by the user	Generated automatically by the system
Role of Vision	Obtain current_state	Obtain current_state and verify classification
Role of LLM	Parse movement instructions	Understand color-sorting semantics
System Capability	Vision-based closed-loop rearrangement	Semantic goal generation + color sorting

Conclusion: the system evolves from executing user-specified positions to understanding task semantics and generating goals automatically.

NEXT STEPS

In the short term, improve demo stability; in the long term, extend toward real hardware and more complex tabletop tasks.

Short-Term Plan

- Improve the algorithm accuracy of demo_6
- Improve the robustness of visual recognition
- Organize experimental logs and results

Long-Term Outlook

- Deploy to a real TM5-700 robot arm
- Extend to a larger discrete workspace
- Use stronger vision models to improve scene understanding
- Continuously update the scene / world state

Future Goal: Enable the robot arm to autonomously perform more complex tabletop manipulation tasks based on natural language, visual perception, and scene state.

https://github.com/Wencheester/tm5_color_pick_demo

Thank you so much!